Real-Time IoT Sensor Data Pipeline

Digital Egypt Pioneers Initiative

Track: Microsoft Data Engineer

ONL5_AIS5_G1 – Team_1

2026

Literature Review

1. Purpose and Scope

This review explains the concepts behind a real-time IoT pipeline, compares the main tool options for each layer, and gives the reason for each choice.

2. Background

2.1 IoT data and why streaming

IoT devices produce continuous, time-stamped readings. Value is highest when readings are acted on quickly (equipment faults, environment alarms), so batch loads that run hourly or daily are often too slow. Sensor data is also messy as devices send duplicates, readings arrive late or out of order, and values can be missing or physically impossible. A pipeline has to handle these cases by design.

2.2 Architecture styles

- **Lambda architecture** runs a batch layer and a speed layer side by side. It gives accurate and fast results but means two code bases to maintain.
- **Kappa architecture** treats everything as a stream over a replayable log. It has one code base, and reprocessing means replaying the log (Kreps, 2014).
- **Medallion layering** (raw, clean, aggregated) organizes stored data by quality level. It fits the project's layered storage and makes replay and audit easy.

The project uses an event-driven, Kappa-style design with layered storage. Kafka is the replayable log, and a single streaming job produces the clean and aggregated layers.

2.3 Core streaming concepts

Concept	Meaning	Why it matters here
Event time vs processing time	When a reading happened vs when the system saw it	Windows must use event time or late data lands in the wrong window (Akidau et al., 2015)
Watermark	The system's estimate of how late data can still arrive	Decides when a window is final and when old state can be dropped
Windowing	Grouping events into time buckets (tumbling, sliding)	Produces per-minute and per-site aggregates for the dashboard

Concept	Meaning	Why it matters here
Delivery semantics	At-most-once, at-least-once, exactly-once	At-least-once delivery plus idempotent writes can give an exactly-once effect, depending on the full persistence and recovery design
Deduplication	Removing repeated events using a unique event ID	Sensors and retries create duplicates
Dead-letter queue	A separate store for records that fail validation	Keeps bad data out of analytics without losing it
Backpressure and consumer lag	A consumer falling behind the producer	An early sign of an overloaded pipeline; a KPI in this project

2.4 Data quality and observability

Streaming data needs checks at ingestion: schema validation, range checks, completeness and freshness. Observability adds metrics on latency, throughput, lag and error rates so that problems are seen before users notice. The project builds both into the pipeline instead of adding them later. Data quality rules are written and reported with Great Expectations.

3. Related Work: Tool Comparison

Each table compares the chosen tool with common alternatives. Statements are general characteristics, and the last column gives the reason for the decision.

3.1 Message broker

Option	Strengths	Limitations for this project	Decision
Apache Kafka	Durable partitioned log, replay within the retention period, high throughput, large ecosystem, works with Spark	Heavier to run than a simple broker	**Chosen:** replay supports reprocessing and the Kappa design; widely used in industry
RabbitMQ	Flexible routing, mature queueing	Classic queues remove messages once acknowledged, so replay is	Not chosen

Option	Strengths	Limitations for this project	Decision
		limited (RabbitMQ Streams add a log model)	
MQTT broker (for example Mosquitto)	Very light, the standard protocol for constrained devices	Not built for long retention or stream processing at scale	Not chosen; could sit in front of Kafka in a real deployment
Azure Event Hubs / IoT Hub	Managed, scalable, Kafka-compatible endpoint (Event Hubs)	Paid cloud service, outside the free-tools rule	Not chosen; design maps to it directly

3.2 Stream processing

Option	Strengths	Limitations for this project	Decision
PySpark Structured Streaming	Python API, SQL-like operations, event-time windows and watermarks, same engine as the course (Armbrust et al., 2018)	Micro-batch model gives latency of seconds, not milliseconds	**Chosen:** fits the track, the team's skills, and the latency target
Apache Flink	Event-at-a-time processing, low latency, strong state handling (Carbone et al., 2015)	Steeper operations and learning curve, not covered in the course	Not chosen
Kafka Streams	Library inside the application, no separate cluster	JVM languages only, so no Python	Not chosen
Azure Stream Analytics	Managed, SQL-like queries	Paid, cloud only	Not chosen

3.3 Storage and serving

Option	Strengths	Limitations for this project	Decision
Apache Parquet	Columnar, compressed, open file format, good for	No transactions or table features by itself; many	**Chosen (confirmed)** for the raw and clean layers; the

Option	Strengths	Limitations for this project	Decision
	raw and clean history and replay	small files need periodic compaction	lake_compaction DAG handles small files
SQL Server Developer	Relational model with keys and constraints, T-SQL, SSMS tooling, in the course scope, free for development and testing	Developer edition is not licensed for production; heavier than lightweight databases	**Chosen (confirmed)** for the serving layer; supports the ER diagram and logical and physical schema in the guidelines
Delta Lake	Adds ACID transactions, schema enforcement and time travel on top of Parquet	Extra dependency and complexity	Not chosen; the lake format is plain Apache Parquet
PostgreSQL	Free open-source relational database	Outside the confirmed stack	Not chosen
TimescaleDB or InfluxDB	Purpose-built for time series	Extra component and different design work	Not chosen
Azure Data Lake or Synapse	Industry-grade cloud analytics	Paid, cloud only	Not chosen

3.4 Dashboard

Option	Strengths	Limitations for this project	Decision
Grafana	Free, strong time-series panels, auto-refresh, built-in Microsoft SQL Server data source, runs in Docker	Another service to configure	**Chosen (confirmed)**
Streamlit	Pure Python, quick to build	Less polished for time-series monitoring	Not chosen

Option	Strengths	Limitations for this project	Decision
Power BI	Familiar in the Microsoft ecosystem	Desktop is free, but live streaming and sharing need paid features	Not chosen

Decision: Grafana, approved on 17 Oct 2026. Streamlit was considered and dropped.

3.5 Data quality

Option	Strengths	Limitations for this project	Decision
Great Expectations	Open-source Python framework with reusable expectation suites, stored validation results and generated data docs; works with Spark and SQL data sources	Built around batch validation, so it validates the stored clean Parquet and SQL Server tables while Spark keeps the per-event checks; setup effort	**Chosen (confirmed)**
Deequ	Library for data unit tests on Spark	Scala-first, Python access through PyDeequ	Not chosen
Soda Core	Open-source checks written in a YAML-style language	Another tool outside the course	Not chosen
Custom rules and a Python runner	Full control and small footprint	Reporting, documentation and upkeep are built by hand	Not chosen

3.6 Orchestration and deployment

Option	Role	Decision
Apache Airflow	Schedules the five required DAGs: stream_health_check, dq_daily_report, lake_compaction, gold_backfill and warehouse_maintenance; in the course scope	Chosen (confirmed)
Docker Compose	Runs the whole stack locally with one command	Chosen (confirmed

Option	Role	Decision
GitHub Actions	Lint and tests on each push	Optional CI

4. Related Architectures and Patterns

- **Cloud reference design:** sensors, IoT Hub or Event Hubs, a stream processor, a data lake and a warehouse, then a BI tool. The project mirrors this with free equivalents, so the design can be moved to the cloud by swapping components.
- **Open-source reference design:** devices, Kafka, Spark or Flink, a lake and a database, then Grafana. This is the design chosen here.
- **Industrial monitoring:** predictive maintenance systems often add machine learning on top of the same pipeline. The project covers the data foundation and leaves models out of scope.

5. Gaps and How the Project Addresses Them

Gap seen in typical tutorials	How the project addresses it
Happy-path data only	Simulator injects late, duplicate and malformed events
No measured results	KPIs for latency, throughput, freshness, errors and uptime, measured and reported
Hard to run elsewhere	One-command Docker Compose setup
Quality checked after the fact	Validation and dead-letter handling inside the stream, plus Great Expectations suites and a daily quality report
Little documentation	Full guideline set: design diagrams, tests, manual, README

6. Summary of Decisions

The confirmed baseline is a Python simulator, Kafka, PySpark Structured Streaming, Apache Parquet for the lake, SQL Server Developer for serving, Great Expectations for data quality, Airflow for orchestration and Docker Compose for local deployment. The dashboard is Grafana. All tools are free to use, fit the course content, and match designs used in industry. Most choices have a cloud equivalent, so the work transfers to Azure.

7. References

Author names, years, venues and page ranges were checked against publisher and index records on 17 Oct 2026. Links were added on 18 Oct 2026 from the publishers' and official project sites.

1. Akidau, T. et al. (2015). The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing. Proceedings of the VLDB Endowment, 8(12), pp. 1792-1803. DOI: 10.14778/2824032.2824076. Page: Google Research.
2. Armbrust, M. et al. (2018). Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark. Proceedings of the ACM SIGMOD International Conference on Management of Data, Houston, pp. 601-613. DOI: 10.1145/3183713.3190664.
3. Carbone, P. et al. (2015). Apache Flink: Stream and Batch Processing in a Single Engine. Bulletin of the IEEE Computer Society Technical Committee on Data Engineering, 36(4), pp. 28-38. Record: TU Delft repository. PDF copy: cs.uic.edu.
4. Kleppmann, M. (2017). Designing Data-Intensive Applications. O'Reilly Media, first edition, ISBN 978-1-449-37332-0. O'Reilly page.
5. Kreps, J. (2014). Questioning the Lambda Architecture. O'Reilly Radar, 2 July 2014. Article.
6. Kreps, J., Narkhede, N. and Rao, J. (2011). Kafka: a Distributed Messaging System for Log Processing. NetDB Workshop, Athens, 12 June 2011. PDF copy.
7. Official documentation: Apache Kafka, Apache Spark Structured Streaming Programming Guide, Apache Parquet, Microsoft SQL Server, Great Expectations (GX Core overview), Apache Airflow, Grafana (Microsoft SQL Server data source), Docker Compose.

Project Planning and Management

1. Project Proposal

1.1 Overview

The project builds an end-to-end real-time data pipeline for IoT sensor data. A device simulator publishes readings (temperature, humidity, CO2, power draw) from a fleet of devices spread over several sites. The readings travel through a message broker, are validated and enriched by a streaming job, are stored in layered storage, and are served to a live dashboard. Bad records are quarantined instead of being lost or crashing the pipeline.

The whole system runs from one Docker Compose command, uses free tools only, and is tested with late, duplicate and malformed events.

1.2 Problem statement

Organizations that monitor equipment and buildings receive thousands of sensor readings per second. Batch processing shows problems hours late, raw data contains duplicates, gaps and corrupt values, and dashboards built on unvalidated data cannot be trusted. The project shows how to deliver validated, fresh sensor data to analysts within seconds, with measurable quality and reliability.

1.3 Objectives

- O1. Ingest simulated sensor events from at least 500 devices at a sustained rate of 500 events per second or more.
- O2. Process events in a streaming job that parses, validates, deduplicates, handles late data with watermarks, and computes windowed aggregates.
- O3. Route invalid events to a dead-letter store with the failure reason, so nothing is silently dropped, and check data quality with Great Expectations suites.
- O4. Store data in layers (raw, clean, aggregated) with Apache Parquet files for the lake, and expose a relational serving layer in SQL Server Developer for analytics.
- O5. Show live device and site metrics on a dashboard with data no more than 15 seconds old.
- O6. Meet the KPI targets in section 5 and prove it with a measured test report.
- O7. Deliver everything the guidelines require: documentation, GitHub repository, automated tests, one-command setup, presentation and infographic.
- O8. Orchestrate maintenance and reporting with Airflow DAGs: stream_health_check, dq_daily_report, lake_compaction, gold_backfill and warehouse_maintenance.

1.4 Scope Management

In scope

- A Python device simulator with configurable rate, device count and fault injection (late, duplicate, malformed, out-of-range events).
- Kafka topics for raw events, with a dead-letter topic.
- PySpark Structured Streaming jobs for validation, deduplication, enrichment and windowed aggregation.
- Layered storage: raw and clean Apache Parquet files, plus a SQL Server Developer serving database.
- A dashboard with live readings, aggregates, device health and data-quality panels.
- Data quality checks with Great Expectations, pipeline monitoring metrics and pytest automated tests.
- Docker Compose deployment, GitHub repository with README, and all documents from the guidelines.
- Airflow orchestration: five DAGs for stream health, daily data-quality reporting, lake compaction, gold-layer backfill and warehouse maintenance.

Out of scope

- Real physical devices or hardware firmware (data is simulated).
- Paid or cloud-only services, including paid Azure streaming services.
- Machine learning or predictive maintenance models (so the infographic has no AI components section).
- Multi-node production clusters, Kubernetes, and high-availability failover.
- Full user authentication system and mobile apps.
- Alert delivery to external channels (email, SMS). Threshold alerts appear on the dashboard only.

1.5 Proposed technology stack

Layer	Tool	Role
Data source	Python simulator	Generates sensor events with fault injection
Messaging	Apache Kafka	Durable, partitioned event log; raw topic keeps 7 days, plus a dead-letter topic
Stream processing	PySpark Structured Streaming	Per-event validation, deduplication, watermarks, windows

Layer	Tool	Role
Lake storage	Apache Parquet files	Raw and clean layers for history and replay
Serving layer	SQL Server Developer	Relational tables for the dashboard and analysts
Data quality	Great Expectations	Expectation suites on the stored clean Parquet and SQL Server tables, validation results, daily quality report
Orchestration	Apache Airflow	Five DAGs: stream_health_check, dq_daily_report, lake_compaction, gold_backfill, warehouse_maintenance
Dashboard	Grafana	Live visualization on the SQL Server serving tables
Testing	pytest	Unit, integration and quality tests
DevOps	Docker Compose (local), GitHub	One-command run, version control

1.6 Expected deliverables

Stage	Deliverables	Due
1. Planning and management	Proposal, plan, roles, risks, KPIs	19 Oct 2026
2. Literature review	Related work and tool comparison, lecturer sections	19 Oct 2026
3. Requirements gathering	Stakeholders, user stories, functional and non-functional requirements	19 Oct 2026
4. System analysis and design	Diagrams, schema, wireframes	6 Nov 2026
5. Implementation	Source code, repository, README, Docker Compose	30 Nov 2026
6 and 7. Testing, final presentation and reports	Test plan and report, technical documentation, user manual, presentation, infographic, optional demo video	4 Dec 2026

1.7 Assumptions and constraints

- Free-to-use tools only, no budget.

- Low resources.
- The pipeline runs on team laptops with Docker, so throughput targets are sized for a laptop.
- All sensor data is simulated, so no personal or sensitive data is involved.
- Everything is uploaded to GitHub.

2. Project Plan

2.1 Milestones

Date	Milestone	Exit criteria
19 Oct 2026	Planning, literature review and requirements submitted	All three documents on GitHub
6 Nov 2026	System analysis and design submitted	All required diagrams, schema and wireframes reviewed
23 Nov 2026	Feature freeze (internal)	End-to-end flow runs; only fixes after this date
30 Nov 2026	Implementation submitted	Code, README and one-command Docker Compose setup on GitHub
4 Dec 2026	Testing, final presentation and reports submitted	Test report, technical docs, manual, presentation, infographic

2.2 Gantt chart

Each block marks a week in which the activity is active. Build activities end by 22 Nov, before the 23 Nov feature freeze.

Activity	19 Oct	17-23 Oct	24-30 Oct	31 Oct-6	7-13 Nov	14-20 Nov	21-30 Nov	1-4 Dec
Planning, literature review, requirements	■							
System analysis and design		■	■	■				
Event contract and interface agreement		■						
Repository, branching, Docker Compose skeleton		■						
Test plan (draft in design, final in testing)				■			■	
Simulator, Kafka producers and topics					■	■		

Activity	19 Oct	17-23 Oct	24-30 Oct	31 Oct-6	7-13 Nov	14-20 Nov	21-30 Nov	1-4 Dec
Spark streaming jobs						■	■	
Great Expectations suites						■	■	
Storage layers (Parquet and SQL Server)						■	■	
Airflow setup and DAGs						■	■	
Dashboard (Grafana)						■	■	
Automated tests and optional CI						■	■	
README and one-command setup						■	■	
Integration, KPI benchmark, bug fixing							■	
Test execution and test report							■	■
Technical documentation and user manual							■	■
Presentation, infographic, optional demo video								■

2.3 GitHub repository

All project files are uploaded to GitHub

Folder	Contents
docs/planning	Planning document and literature review
docs/requirements	Requirements document
docs/design	All diagrams, schema, wireframes
src	Producers, streaming jobs, serving layer, dashboard, Airflow DAGs, Great Expectations suites
tests	Automated tests and data quality checks
docs/testing	Test plan, test report, bug reports
docs/final	Technical documentation, user manual, presentation, infographic, demo video or link
Repository root	README.md (installation, configuration, execution), docker-compose.yml, .env.example without credentials, .gitignore

3. Task Assignment and Roles

Member	Role	Responsibilities	Main deliverables	Backup
Member 1	Team lead, documentation, presentation	Planning, coordination, submissions, README draft, final review	Proposal, Gantt plan, roles table, problem statement, presentation, infographic	Member 4
Member 2	Kafka and simulator	Device simulator, Kafka topics and producers, related-work note	Simulator, topics, sequence and activity diagrams, literature review, optional demo video	Member 3
Member 3	PySpark and data quality	Streaming jobs, validation, deduplication, windows, Great Expectations suites, KPI measurement	Streaming code, DFD, architecture (with Member 2), KPI definitions	Member 2
Member 4	Storage, ER and dashboard	Requirements, storage layers, SQL Server schema, dashboard	Requirements document, use case diagram, ER diagram and schema, wireframes, dashboard, technical documentation, user manual	Member 1
Member 5	QA, DevOps and GitHub	Repository, Docker Compose, Airflow deployment, tests, CI, risk register	Risk register, repo structure, deployment and component diagrams, test plan, test report	Member 3

Working agreements

- One feature branch per task, merged through a pull request.
- Meaningful commit messages and PR descriptions.

- Weekly sync, plus a short check-in at each interface point (simulator to Kafka, Kafka to Spark, Spark to storage, storage to dashboard).
- Interfaces are fixed early: event schema, topic names, table names. Changes go through an issue and a PR.
- Blockers are raised the same day in the team channel.

4. Risk Assessment and Mitigation

Likelihood (L) and impact (I) are rated Low, Medium or High. Member 5 keeps this register and reviews it at each weekly sync.

ID	Risk	L	I	Mitigation	Owner
R1	Data loss between broker, processor and storage	Medium	High	Kafka replication settings and offsets committed after successful write; checkpointing in Spark; raw layer kept for replay; completeness check comparing counts	Member 3
R2	Consumer lag grows under load	Medium	High	Partition topics by device; tune batch interval and shuffle partitions; monitor lag; run load tests before the freeze	Member 3
R3	Schema changes break the pipeline	Medium	Medium	Versioned event schema with explicit fields; schema validation at entry; unknown sensor types and missing required fields routed to the dead-letter queue, extra fields dropped and counted; schema changes only through PR	Member 2
R4	Tool setup problems on different laptops	High	Medium	Docker Compose with pinned image versions; setup guide; early environment test in week 1; shared troubleshooting notes	Member 5
R5	Laptops lack resources for the full stack	Medium	Medium	Resource limits per container; reduced demo profile; scaled-down simulator settings	Member 5

ID	Risk	L	I	Mitigation	Owner
R6	Duplicate or late events corrupt aggregates	High	Medium	Deduplicate on event ID with a watermark; event-time windows; late-event tests	Member 3
R7	Integration problems discovered late	Medium	High	Fixed interfaces in week 1; thin end-to-end slice early; feature freeze on 23 Nov	Member 1
R8	Scope creep (alerting, extra dashboards)	Medium	Medium	Out-of-scope list; extras only after KPIs are met; lead approves additions	Member 1
R9	Team member unavailable	Medium	Medium	Backup owner per role; documented handoffs; small tasks with clear acceptance criteria	Member 1
R10	Missed deadline for a document stage	Low	High	Weekly schedule with owners; draft one week before each due date; lead reviews before submission	Member 1
R11	Dashboard shows stale or wrong data	Medium	Medium	Freshness metric on the dashboard; data quality panel; reconcile dashboard numbers with serving tables	Member 4
R12	Repository mess (large files, secrets, bad history)	Low	Medium	.gitignore, no data dumps or credentials committed, environment files via examples, PR reviews	Member 5
R13	SQL Server Developer container is memory-hungry next to Kafka, Spark and Airflow	High	Medium	Set memory limits per container; test the full stack early on every laptop; keep a reduced demo profile; Developer edition only for development and demo	Member 5

ID	Risk	L	I	Mitigation	Owner
R14	The starting 2-minute watermark may suit the data badly (unstable aggregates or too much state)	Medium	Medium	Test several lateness thresholds with the simulator before the feature freeze; record the results; the value changes only through a pull request with test evidence	Member 3
R15	Airflow deployment complexity delays the five required DAGs	Medium	Medium	Pinned Airflow image in Docker Compose; set up early; build the simplest DAG first (stream_health_check) and add the others one by one	Member 5
R16	Great Expectations does not fit a streaming flow directly	Medium	Medium	Great Expectations validates the stored clean Parquet and SQL Server tables (decision D-09); fast structural checks stay in Spark; run an early spike to confirm the setup	Member 3
R17	Closed decisions drift (dashboard tool, event contract, ranges, watermark)	Medium	High	Decision log in section 6; any change goes through an issue and a pull request and is recorded with its date	Member 1

5. KPIs

Targets are initial values. They are validated in the benchmark run in the last implementation week and reported in the test report.

KPI	Definition and measurement boundary	Target	Acceptance test
Processing latency	Time from Kafka ingest time to served time (row queryable in SQL Server). Event-time delay (event time to ingest time) is reported separately, and injected late events are excluded	p95 of 10 s or less; median of 5 s or less	10-minute normal-load run; percentiles computed from stored timestamps
Throughput	Valid events processed per second by the streaming job; rejected and duplicate events counted separately	500 events/s sustained; 1,000 events/s peak	10-minute sustained run and a peak run; pass if the target rate is met and consumer lag does not keep growing; record hardware and container limits
Data freshness	Current UTC time minus the served time of the newest successfully served event per site, on one host clock. A device that stops sending is shown as silent, not as stale pipeline data	15 s or less	Dashboard panel checked during a normal-load run with timestamped screenshots
Error or dead-letter rate	Share of injected invalid events captured with a reason, and share of valid events lost to pipeline failures after retries	100% of injected invalid events captured; under 0.1% pipeline-caused failures	Fault-injection run; compare injected counts with dead-letter counts
Pipeline uptime (availability)	Share of health-check samples (every 10 s) in which all required services are healthy: Kafka, streaming job, SQL Server, Airflow and dashboard	99% or more over a 2-hour soak	2-hour soak test; recovery time measured separately for each service restart

KPI	Definition and measurement boundary	Target	Acceptance test
Recovery time	Time from a service restart until processing resumes with no data loss	60 s or less per tested service	Restart each of Kafka, the streaming job and SQL Server once during a run
Data completeness (extra)	Valid unique events produced by the simulator versus events stored in the clean layer and SQL Server; intentional duplicate suppression is excluded	100%	Reconciliation query after each test run, covering retried writes
Duplicate rate (extra)	Duplicate event IDs in the clean layer and the serving table	0	SQL and Parquet duplicate check
Consumer lag (extra)	Age of the oldest unprocessed message (now minus its Kafka timestamp), tracked beside lag in messages	5 s or less under normal load	Kafka consumer group metrics during a normal-load run

Measurement plan

- Every record carries four timestamps in UTC: event time, ingest time (Kafka append time), processed time and served time.
- All services run on one host clock, so timestamp differences are comparable.
- Latency, throughput, freshness and lag are computed from these timestamps and shown on a monitoring panel.
- The benchmark runs three scenarios: normal load, peak load, and a fault-injection run with late, duplicate and malformed events.
- Results go into the test report with the machine specification used.

Requirements Gathering

This document covers the four Requirements Gathering deliverables from the project guidelines: stakeholder analysis, user stories and use cases, functional requirements, and non-functional requirements. Priorities use MoSCoW (Must, Should, Could). Targets match the KPIs in the Planning document.

1. Stakeholder Analysis

Stakeholder	Role	Needs from the system	Influence	Interest
Data engineers (the team)	Build, run and maintain the pipeline	Clear setup, monitoring metrics, replay of raw data, simple debugging	High	High
Data analysts	Query the clean and aggregated data	Trusted, deduplicated data in a relational schema, documented columns	Medium	High
Dashboard consumers (operations or facility managers)	Watch device and site status	Live readings, device health, clear data-quality indicators	Medium	High
Device and field technicians	Respond to faults	Clear view of offline devices and out-of-range readings	Low	Medium
Lecturers and assessors	Evaluate the project	Complete documents, working demo, measured KPIs, clean GitHub repository	High	High
Platform and DevOps support (the team)	Deploy and operate the stack	One-command setup, pinned versions, health checks	Medium	Medium

2. User Stories and Use Cases

2.1 User stories

ID	As a	I want	So that	Related FR	Priority
US-01	Data engineer	to start the whole pipeline with one command	anyone can run it without manual setup	FR-21	Must
US-02	Data engineer	to simulate many devices with configurable rate and fault injection	I can test normal and bad-data conditions	FR-01, FR-02	Must
US-03	Data engineer	invalid events sent to a dead-letter store with the reason	bad data is never lost or silently dropped	FR-08, FR-09	Must
US-04	Data engineer	duplicate events removed	counts and averages are correct	FR-10	Must
US-05	Data engineer	late events handled by the approved watermark policy (2-minute start value)	windows stay correct without unbounded state	FR-11	Must
US-06	Data engineer	raw events kept unchanged	I can replay and reprocess after a bug fix	FR-12	Should
US-07	Data analyst	clean readings in a relational table with keys	I can query them with SQL	FR-13, FR-14	Must
US-08	Data analyst	per-minute and per-site aggregates	I can analyze trends quickly	FR-15	Must
US-09	Dashboard consumer	to see device readings within seconds of arrival	I can react to what is happening now	FR-16	Must
US-10	Dashboard consumer	to see which devices are offline or silent	I know where data is missing	FR-17	Should
US-11	Dashboard consumer	a data-quality panel showing rejected and late events	I know how far to trust the numbers	FR-18	Should
US-12	Operations manager	threshold alerts shown on the dashboard	I notice dangerous readings fast	FR-19	Could
US-13	Data engineer	latency, throughput and lag metrics	I can prove the KPIs are met	FR-20	Must

ID	As a	I want	So that	Related FR	Priority
US-14	Assessor	tests, documents and a README in GitHub	I can verify the work	FR-22, FR-23, FR-28	Must
US-15	Data engineer	data-quality rules defined as Great Expectations suites with a daily report	quality is checked repeatably and visible to everyone	FR-24	Must
US-16	Data engineer	Airflow DAGs for stream health, daily quality report, lake compaction, gold backfill and warehouse maintenance	routine operations run on schedule and on demand	FR-25	Must
US-17	Data engineer	failed writes retried without creating duplicates	no valid event is lost or stored twice	FR-26, FR-27	Must

2.2 Use cases

The use case diagram is part of the Design stage. These are the use cases it will show.

ID	Use case	Actor	Summary
UC-01	Generate sensor events	Simulator (system actor), data engineer	Simulator creates events at a configured rate, with optional faults, and publishes to Kafka
UC-02	Validate and clean events	Streaming job	Parses events, checks schema and ranges, removes duplicates, routes bad events to the dead-letter topic
UC-03	Aggregate readings	Streaming job	Computes windowed aggregates per device and per site using event time
UC-04	Store and serve data	Streaming job, SQL Server	Writes raw, clean and aggregated layers and updates serving tables
UC-05	View live dashboard	Dashboard consumer	Opens the dashboard and sees live readings, device health and quality indicators
UC-06	Query historical data	Data analyst	Runs SQL on the clean and aggregate tables

ID	Use case	Actor	Summary
UC-07	Monitor pipeline health	Data engineer	Reads latency, throughput, lag and error metrics
UC-08	Inspect and replay rejected or raw data	Data engineer	Reviews dead-letter records and replays raw data after a fix
UC-09	Run scheduled operations	Airflow, data engineer	Airflow runs stream health checks, the daily quality report, lake compaction, gold backfill and warehouse maintenance on schedule or on demand

UC-02 in detail (example)

Field	Description
Actor	Streaming job
Precondition	Raw events are available on the Kafka topic
Trigger	A micro-batch of events arrives
Main flow	1. Parse the event. 2. Check required fields, types and value ranges. 3. Drop duplicates by event ID within the watermark. 4. Write valid unique events to the clean layer and the serving table. 5. Verify the required writes
Alternate flow: invalid	Parsing, schema or range check fails: send the original payload and the reason to the dead-letter topic, then continue
Alternate flow: duplicate	The event is suppressed from the clean and serving layers and counted
Alternate flow: late	Events later than the 2-minute watermark go to the late-events Parquet path and are corrected by the gold_backfill DAG
Alternate flow: infrastructure failure	The write is retried with idempotent writes; the event is not counted as persisted until the required writes are verified
Postcondition	Every event is accounted for as stored, rejected to the dead-letter store, suppressed as a duplicate,

Field	Description
	handled as late under the approved policy, or still being retried

3. Functional Requirements

ID	Area	Requirement	Priority
FR-01	Ingestion	The simulator shall generate events for a configurable number of devices and sites at a configurable rate	Must
FR-02	Ingestion	The simulator shall inject late, duplicate, malformed and out-of-range events at configurable percentages	Must
FR-03	Ingestion	Each event shall carry a unique event ID, device ID, site ID, sensor type, value, unit and event timestamp	Must
FR-04	Ingestion	Events shall be published to a Kafka topic partitioned by device ID	Must
FR-05	Ingestion	A separate Kafka dead-letter topic shall receive rejected events	Must
FR-06	Processing	The streaming job shall consume the raw topic and parse events against a defined schema	Must
FR-07	Processing	The job shall enrich events with device and site reference data	Should
FR-08	Processing	The job shall validate required fields, types and value ranges per sensor type	Must
FR-09	Processing	A consumer shall persist records from the dead-letter topic to durable dead-letter storage with the failure reason and original payload	Must
FR-10	Processing	The job shall remove duplicate events by event ID	Must
FR-11	Processing	The job shall use event-time windows with a watermark to handle late events; the watermark starts at 2 minutes and changes only through a pull request with test evidence, and later events go to the late-events Parquet path and are corrected by gold_backfill	Must
FR-12	Storage	Raw events shall be stored unchanged in the raw Parquet layer; the Kafka raw topic keeps 7 days and	Should

ID	Area	Requirement	Priority
		the raw Parquet layer is kept for the whole project, so replay after 7 days uses Parquet	
FR-13	Storage	Validated events shall be stored in the clean Parquet layer and in a SQL Server Developer table	Must
FR-14	Storage	The serving schema shall use primary and foreign keys for devices, sites and readings	Must
FR-15	Storage	Windowed aggregates (average, minimum, maximum, count) shall be stored per device and per site	Must
FR-16	Serving	The Grafana dashboard shall show live readings and aggregates, refreshing at an interval no longer than the freshness target	Must
FR-17	Serving	The dashboard shall show device status (active or silent) based on last event time	Should
FR-18	Serving	The dashboard shall show data-quality indicators: rejected, duplicate and late event counts	Should
FR-19	Alerting	The dashboard shall highlight readings that cross configured thresholds	Could
FR-20	Operations	The pipeline shall record event, ingest, processed and served timestamps and expose latency, throughput and consumer lag	Must
FR-21	Operations	The full stack shall start with one Docker Compose command and stop cleanly	Must
FR-22	Quality	The project shall include pytest unit tests and data quality checks	Must
FR-23	Documentation	The repository shall include a README with installation, configuration and execution steps	Must
FR-24	Quality	Data quality rules shall be defined as Great Expectations expectation suites that validate the stored clean Parquet and SQL Server tables, run on a schedule and on demand, with stored validation results and a daily report	Must
FR-25	Orchestration	Airflow shall run five DAGs: stream_health_check, dq_daily_report, lake_compaction, gold_backfill and warehouse_maintenance	Must

ID	Area	Requirement	Priority
FR-26	Reliability	Failed writes shall be retried, writes shall be idempotent so retries create no duplicates, and an event counts as persisted only after the required writes are verified	Must
FR-27	Operations	A reconciliation check shall compare produced, stored, rejected and duplicate-suppressed event counts	Should
FR-28	Documentation	All project files (documents, diagrams, source code, Airflow DAGs, Great Expectations suites, tests, reports, presentation, infographic) shall be uploaded to the GitHub repository using the folder structure in the Planning document, section 2.6	Must

Event data requirements

Field	Type	Rule
event_id	string	Required; UUID created by the producer, globally unique, reused when a send is retried
device_id	string	Required; must exist in the device registry
site_id	string	Required; must match the site registered for the device, otherwise dead-letter
sensor_type	string	One of temperature, humidity, co2, power; unknown types go to the dead-letter queue
value	double	Required, not null; temperature -40 to 85 °C, humidity 0 to 100 %, CO2 0 to 5,000 ppm, power 0 to 5,000 W
unit	string	Matches the sensor type (°C, %, ppm, W)
event_ts	timestamp	Required; ISO 8601 in UTC with Z; rejected if more than 5 s in the future

Other rules: missing or null required fields go to the dead-letter queue, and extra fields are dropped and counted. Planned simulation size: 500 devices sending one reading per second, which is about 500 events per second, with peak tests at 1,000 events per second.

Acceptance evidence

Each Must requirement needs a testable outcome. These are suggested evidence standards; the project guidelines and lecturer instructions remain authoritative. Test IDs are assigned in the test plan.

Requirements	Suggested acceptance evidence	Test ID
FR-01 to FR-06	Simulator configuration, Kafka topic configuration (raw and dead-letter) and a measured production-rate test	To be assigned
FR-08, FR-09	Test fixtures showing invalid records captured with correct rejection reasons	To be assigned
FR-10	SQL and Parquet checks showing no duplicate event IDs in the clean and serving layers	To be assigned
FR-11	Tests for events inside and outside the 2-minute watermark	To be assigned
FR-13 to FR-15	Database schema, successful writes and verified aggregate calculations	To be assigned
FR-16 to FR-20	Dashboard screenshots plus timestamped latency, freshness and throughput measurements	To be assigned
FR-21	Clean-start test using the documented Docker Compose command	To be assigned
FR-22, FR-24	Automated test results and the Great Expectations validation report	To be assigned
FR-23, FR-28	README walkthrough completed by a team member other than its author; repository checked against the folder structure with every deliverable linked	To be assigned
FR-25	Each of the five DAGs run on schedule and on demand with logged results	To be assigned
FR-26, FR-27	Forced write failure and service restart showing retries without duplicates, and a passing reconciliation	To be assigned

4. Non-Functional Requirements

ID	Category	Requirement	Target	Priority
NFR-01	Performance	Processing latency from Kafka ingest time to the serving layer, with event-time delay reported separately	p95 of 10 s or less	Must
NFR-02	Performance	Sustained throughput on a laptop	500 events/s, with a 1,000 events/s peak test	Must
NFR-03	Performance	Dashboard data age	15 s or less	Must
NFR-04	Reliability	No data loss between broker and storage	100% of valid, unique events stored (intentional duplicate suppression excluded)	Must
NFR-05	Reliability	Pipeline availability and recovery	99% over a 2-hour soak; restart recovery within 60 s	Should
NFR-06	Reliability	Failed records handled without stopping the job	Zero job crashes from bad records	Must
NFR-07	Scalability	Throughput grows by adding Kafka partitions and Spark resources	Documented scaling path; tested with at least two partition counts, recording partition counts, resources, test duration and throughput and lag results	Should
NFR-08	Data quality	Duplicates in clean and serving layers	0	Must
NFR-09	Data quality	Invalid injected events captured in the dead-letter store	100% with a reason	Must

ID	Category	Requirement	Target	Priority
NFR-10	Security	No credentials in the repository; secrets in environment files not committed; services reachable only on the local network	Checked in code review	Must
NFR-11	Maintainability	Modular, commented code with consistent naming and PR review	Every merge through a reviewed PR	Should
NFR-12	Portability	Runs on Windows, macOS or Linux with Docker	Setup tested on at least two operating systems	Should
NFR-13	Usability	A new user can start the pipeline and read the dashboard from the manual	Setup in 15 minutes or less	Should
NFR-14	Observability	Pipeline health visible without reading logs	Metrics panel on the dashboard	Should

7. Traceability

Objective (Planning document)	Requirements
O1 Ingest at 500 events/s	FR-01 to FR-05, NFR-02
O2 Streaming validation, deduplication, windows	FR-06 to FR-11, NFR-06, NFR-08
O3 Dead-letter handling	FR-05, FR-09, FR-24, NFR-09
O4 Layered storage and relational serving	FR-12 to FR-15, FR-26
O5 Live dashboard	FR-16 to FR-19, NFR-03
O6 Measured KPIs	FR-20, NFR-01 to NFR-05
O7 Full guideline deliverables	FR-21 to FR-23, FR-28, NFR-10 to NFR-14
O8 Orchestration with Airflow	FR-25, FR-27